



**UNIVERSIDADE FEDERAL DE
MINAS GERAIS**

TRABALHO FINAL DE AUTOMAÇÃO EM TEMPO REAL

Entrega 1: Definição da Arquitetura

Ana Clara Melado, 2023421386

Bruno Araujo Pinto, 2023038965

Mariana Villefort Rabelo, 2022061033

Belo Horizonte
18 de maio de 2026

1 Introdução

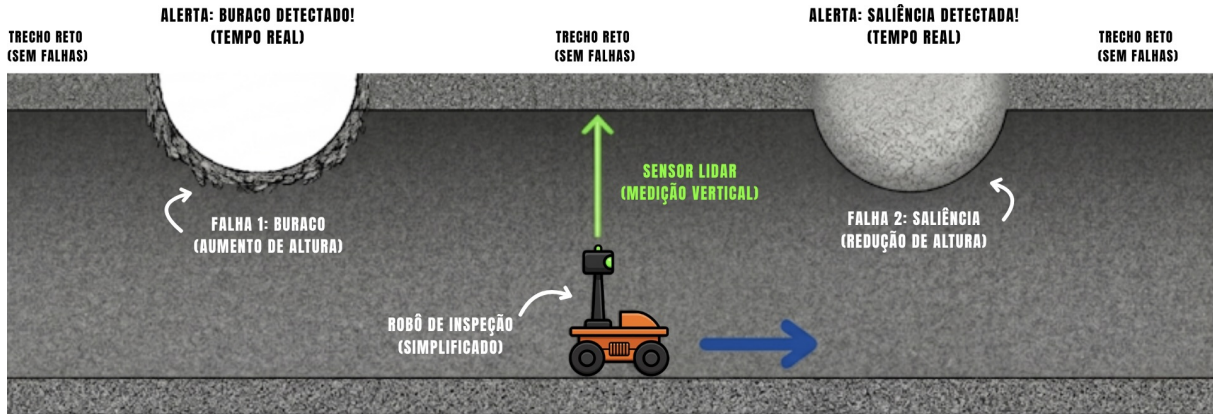


Figura 1: Túnel (vista lateral 2D)

A inspeção periódica de túneis e estruturas civis é fundamental para garantir a segurança operacional e estrutural desses ambientes. Entretanto, métodos tradicionais de inspeção manual apresentam limitações relacionadas ao custo, ao tempo de execução e à exposição de operadores a ambientes potencialmente perigosos. Nesse contexto, sistemas autônomos de inspeção vêm sendo cada vez mais utilizados, permitindo a coleta contínua de dados e a identificação precoce de falhas estruturais.

Este trabalho tem como objetivo o desenvolvimento da arquitetura de um sistema embarcado para inspeção automática de túneis, utilizando conceitos de Automação em Tempo Real, programação concorrente e sincronização entre tarefas. O sistema proposto simula o funcionamento de um robô autônomo responsável por percorrer um túnel realizando medições da superfície do teto por meio de sensores simulados, como encoder e LIDAR, possibilitando a detecção de irregularidades estruturais.

A implementação desenvolvida na primeira etapa do projeto concentra-se principalmente nas tarefas responsáveis pelo controle de navegação, cálculo de distância percorrida, reconstrução da superfície do túnel e coleta de dados. Para isso, foram utilizados processos e múltiplas threads em C++, além de mecanismos de sincronização como `mutex` e `condition_variable`, garantindo acesso seguro aos buffers compartilhados e evitando condições de corrida durante a troca de informações entre produtor e consumidor.

Além da implementação dos buffers compartilhados, o projeto também realiza testes preliminares de sincronização e bloqueio, verificando situações de buffer vazio e buffer cheio. Esses testes permitem validar o comportamento concorrente do sistema e demonstrar a correta coordenação entre as tarefas executadas em paralelo, aspecto essencial em aplicações de tempo real.

Por fim, esta etapa do trabalho estabelece a base arquitetural necessária para as próximas implementações do sistema completo, incluindo comunicação remota, interface gráfica de operação e integração com protocolos de comunicação.

2 Proposta de Arquitetura

A arquitetura do trabalho foi realizada com o intuito de facilitar o máximo possível a aplicação dos conceitos, por isso foi priorizada a utilização de threads à criação de proces-

so. A partir dos requisitos mínimos, 2 processos foram criados. Além disso, aproveita-se das threads originárias dos próprios processos para evitar a criação desnecessária de mais threads. A arquitetura pode ser conferida na figura 2.

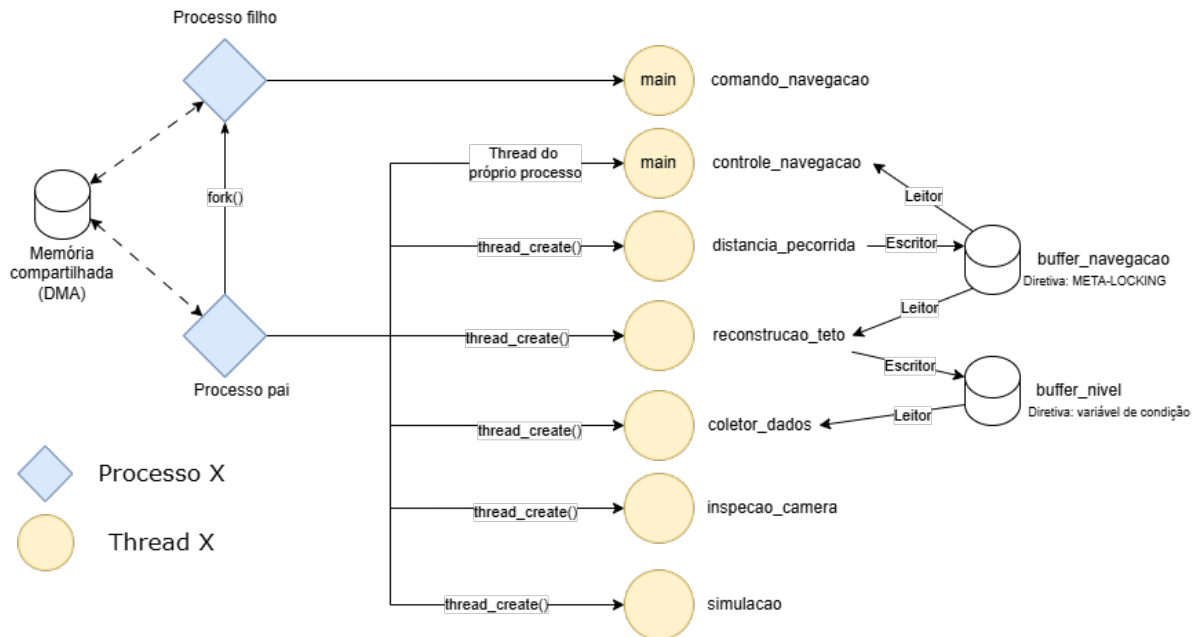


Figura 2: Proposta de arquitetura para o trabalho prático. Losangos representam processos e círculos representam threads.

Ademais, para a comunicação e para os recursos compartilhados, foi utilizada uma diretiva diferente de sincronização. No caso do `buffer_navegacao`, foi utilizado o método do META-LOCKING. Para o `buffer_nivel`, foram utilizadas variáveis de condição, de forma mais simples, usando `wait` e `signal` diretamente, visando simplificar a implementação do meta-locking, com condições complexas de acesso ao recurso compartilhado. E por fim, a comunicação entre os processos foi feita por meio da memória compartilhada.

A implementação dos buffers compartilhados foi realizada por meio de `std::vector<float>` e operações módulo (`idx % ELEMENTOS_BUFFER`). Dessa forma, garante-se a condição circular por meio do próprio tratamento da biblioteca `vector` e da aritmética simples. Foi escolhido o tipo `float`, pois trata-se de fenômenos físicos como velocidade e posição. Por fim, os buffers possuem tamanho inicial de 10 elementos (`ELEMENTOS_BUFFER`), o que se mostra um valor razoável para ações em tempo real.

2.1 Escolha da sincronização

A lógica de sincronização foi implementada de forma diferente para cada buffer devido à natureza dos acessos realizados sobre cada recurso compartilhado. No caso do `BUFFER_NAVEGACAO`, existe uma tarefa escritora, associada ao cálculo da distância percorrida, e mais de uma tarefa leitora, associada ao controle de navegação e à reconstrução do teto. Por esse motivo, foi adotada uma lógica de leitores e escritores baseada em meta-locking, utilizando contadores auxiliares para indicar leitores ativos, leitores aguardando, escritores ativos e escritores aguardando. Antes de acessar o buffer, cada tarefa verifica o estado desses contadores dentro de uma região protegida por `mutex`. Caso o recurso não esteja disponível, a tarefa é bloqueada por meio de uma `condition_variable`,

permanecendo em espera até que outra tarefa libere o acesso e envie uma notificação.

Essa estratégia evita situações de *race condition*, nas quais duas ou mais threads acessariam simultaneamente o mesmo recurso compartilhado de forma inconsistente, além de dar prioridade aos escritores (como pode ser visto nas condições do meta-locking), evitando inanição. No caso do `BUFFER_NAVEGACAO`, a sincronização foi feita com variáveis de condição, que impedem que uma escrita ocorra enquanto há leitores ativos e também impede que outros leitores ou escritores acessem durante uma escrita ativa. Dessa forma, evita-se que uma thread leia um valor parcialmente atualizado ou que uma escrita sobrescreva dados enquanto outra tarefa ainda está utilizando o buffer.

Já o `BUFFER_NIVEL` foi implementado com uma lógica produtor-consumidor, pois possui uma relação mais direta entre as tarefas: a reconstrução do teto atua como produtora das amostras de nível, enquanto o coletor de dados atua como consumidor. Quando o `BUFFER_NIVEL` está vazio, ou seja, quando `dados_nivel == 0`, a tarefa consumidora é bloqueada por uma variável de condição até que a tarefa produtora escreva um novo dado e envie uma notificação. Isso evita leitura inválida de posições sem dados disponíveis. De forma análoga, quando o buffer atinge sua capacidade máxima, a tarefa produtora é bloqueada até que o consumidor leia uma amostra e libere espaço. Esse mecanismo evita sobrescrita de dados ainda não consumidos.

O uso de `condition_variable` também evita espera ativa, pois as threads bloqueadas não ficam executando continuamente testes em laços de repetição enquanto aguardam o recurso. Em vez disso, elas são suspensas pelo sistema operacional e retomadas apenas quando uma condição relevante é sinalizada. Isso reduz o uso desnecessário de CPU e torna a execução mais adequada para aplicações concorrentes e de tempo real.

Por fim, a utilização de `mutexes`, variáveis de condição e contadores de controle contribui para evitar erros clássicos de programação concorrente, como *race conditions*, leituras inconsistentes, sobrescrita de dados, perda de eventos, inanição e bloqueios permanentes.

3 Resultados de Testes Preliminares Realizados

Durante a fase de desenvolvimento da arquitetura do sistema, foram realizados testes preliminares para validar a implementação dos buffers compartilhados e a sincronização entre as tarefas que escrevem e leem nos buffers. Esses testes foram fundamentais para garantir que o sistema se comporta corretamente em diferentes condições do sistema, evitando, por exemplo, condições de corrida, inanição e garantindo a integridade dos dados e a execução e finalização corretas de todas as partes já implementadas do código.

Os testes foram conduzidos com dados simulados, uma vez que o restante do código ainda será realizado na etapa posterior do projeto. Foram feitos testes de execução, de diferentes quantidades de falhas encontradas no caminho percorrido, casos onde o buffer estava vazio, garantindo que as tarefas consumidoras aguardavam corretamente até que os dados estivessem disponíveis, além de casos de bloqueio de leitores e escritores do buffer de navegação, garantindo a boa execução do meta-locking implementado, em que mais de uma thread pode ler o buffer, porém somente uma thread pode acessá-lo em caso de escrita.

Os resultados desses testes preliminares foram positivos, demonstrando que a implementação dos buffers compartilhados e a sincronização entre as tarefas estão funcionando conforme o esperado. Esses resultados fornecem uma base sólida para a próxima etapa do desenvolvimento do sistema, onde será finalizada a implementação do sistema.

3.1 Compilação e Execução Inicial

Inicialmente, foi validado o processo de compilação do programa por meio do arquivo *Makefile*, que foi comprovada pela utilização de logs que imprimem mensagens indicando tarefas que foram realizadas. Durante a execução inicial, verificou-se a criação do processo associado ao comando de navegação, a escrita de comandos em memória compartilhada e a leitura desses comandos pelo processo principal. Foram observados valores coerentes para o comando de operação automática e para o setpoint de velocidade, indicando que a estrutura inicial de compartilhamento de informações foi inicializada corretamente.

Em seguida, foram criadas as threads correspondentes às principais tarefas do sistema: cálculo de distância percorrida, inspeção por câmera, coletor de dados, reconstrução do teto e controle de navegação. A criação das threads foi acompanhada por mensagens de log, permitindo verificar a inicialização e execução das tarefas.

Após isso, foram impressos diversos logs da execução, que serão melhor explorados nos próximos testes.

3.2 Teste sem Detecção de Falhas

O primeiro teste funcional teve como objetivo verificar o comportamento do sistema sem que houvesse detecção de falhas na superfície do teto. Nesse caso, a tarefa de reconstrução do teto produziu números simulados de distância (que futuramente virão do LIDAR) para o `BUFFER_NIVEL`, enquanto o coletor de dados realizou a leitura dessas informações.

Durante a execução, observou-se que não houve acionamento indevido da câmera nem redução de velocidade associada à inspeção, o que indica que a lógica de falha permaneceu inativa durante o teste. Ao final da execução, todas as threads foram finalizadas corretamente e os buffers finais foram impressos, demonstrando que a ausência de falhas não provocou bloqueio permanente no sistema.

3.3 Teste com uma Falha Detectada

Esse teste teve como objetivo validar o comportamento do sistema diante da detecção de uma única falha na superfície do teto. Para isso, a lógica de detecção da tarefa de reconstrução foi configurada para gerar um evento de falha em apenas uma iteração durante toda a execução.

Durante o teste foi registrada a detecção de falha em um único instante da execução, seguida do acionamento lógico da câmera e da redução do setpoint de velocidade. Após o tratamento do evento, a execução prosseguiu normalmente.

Ao final, todas as threads foram encerradas sem travamentos, indicando que o sistema foi capaz de tratar uma falha isolada sem comprometer a execução das demais tarefas.

Além disso, foi possível verificar a tarefa de inspeção por câmera a partir da geração de eventos de falha pela tarefa de reconstrução do teto. Verificou-se que a câmera não executava atividades enquanto não havia falhas detectadas. Quando uma falha foi simulada, a reconstrução incrementou o contador de eventos de câmera, atualizou os estados compartilhados e notificou a variável de condição associada à inspeção.

Durante os testes, foram registradas mensagens indicando o início e a finalização da inspeção por câmera. Esse comportamento confirmou que a thread da câmera foi corretamente desbloqueada diante de eventos de falha e que retornou ao estado inativo após o processamento.

3.4 Teste com Múltiplas Falhas

Outro teste realizado foi o de múltiplas falhas, no qual a tarefa de reconstrução do teto foi configurada para simular a detecção de falha em todas as iterações. O objetivo desse teste foi verificar se o sistema seria capaz de lidar com acionamentos sucessivos da lógica de inspeção.

Durante a execução, foram registradas múltiplas mensagens de falha detectada, cada uma associada ao acionamento lógico da câmera e à redução do setpoint de velocidade. Mesmo com sucessivos eventos de inspeção, o sistema continuou realizando a escrita e leitura dos dados no buffer de nível sem apresentar erros.

Esse teste permitiu validar o funcionamento da câmera sem perdas de eventos quando múltiplas falhas são detectadas em sequência. Ao final da execução, a thread de reconstrução foi finalizada, a thread de câmera encerrou corretamente e todas as demais tarefas concluíram sua execução. Dessa forma, o teste indicou que o sistema é capaz de tratar múltiplos eventos de falha sem bloqueio permanente.

3.5 Teste de Buffer de Nível Vazio

O teste de buffer de nível vazio teve como objetivo verificar se uma tarefa consumidora permanece bloqueada quando não há dados disponíveis para leitura. Esse comportamento foi observado na tarefa coletora de dados, responsável por consumir informações do `BUFFER_NIVEL`.

Durante a execução, o coletor registrou mensagens indicando que o buffer estava vazio e permaneceu aguardando a chegada de novos dados. Após a tarefa de reconstrução do teto escrever um novo dado e notificar a variável de condição correspondente, o coletor retomou a execução e realizou a leitura do dado disponível.

Esse comportamento demonstra que o consumidor não realiza leituras inválidas quando o buffer está vazio. Em vez disso, a thread permanece bloqueada, sem espera ativa, até que o produtor disponibilize uma nova amostra. Assim, o teste validou o tratamento correto da condição de buffer vazio.

3.6 Teste de Buffer de Nível Cheio

O teste de buffer de nível cheio foi realizado com o objetivo de verificar se a tarefa produtora é corretamente bloqueada quando o buffer atinge sua capacidade máxima. Para isso, a tarefa de reconstrução do teto foi configurada para produzir dados em uma taxa superior à taxa de consumo do coletor de dados.

Durante a execução, o `BUFFER_NIVEL` foi sendo ocupado a cada iteração até atingir sua capacidade máxima. Nesse momento, a tarefa de reconstrução registrou a condição de buffer cheio e permaneceu bloqueada aguardando a liberação de espaço.

Quando o coletor de dados consumiu as amostras (como a execução é muito rápida, na figura todas foram consumidas antes da sinalização), a tarefa de reconstrução foi notificada e retomou a escrita no buffer. Esse comportamento confirma que o produtor não sobrescreve dados quando o buffer está cheio, permanecendo em espera até que o consumidor libere espaço. Assim, a lógica produtor-consumidor implementada para o buffer de nível foi validada.

3.7 Teste de Bloqueio de Leitores e Escritores

Foi realizado também um teste específico para o `BUFFER_NAVEGACAO`, cujo acesso é controlado por uma lógica de leitores e escritores usando meta-locking. Esse buffer é escrito pela tarefa de cálculo de distância percorrida e lido pelas tarefas de controle de navegação e reconstrução do teto.

Para validar o mecanismo de sincronização, foram introduzidos atrasos artificiais nas regiões críticas de leitura e escrita. Inicialmente, a tarefa de distância percorrida manteve o lock de escrita com o escritor por um intervalo maior, fazendo com que as tarefas leitoras aguardassem a liberação do recurso. Os logs indicaram que tanto a reconstrução do teto quanto o controle de navegação permaneceram bloqueados enquanto havia um escritor ativo.

Em seguida, observou-se também a situação inversa: a tarefa escritora aguardou a liberação do buffer enquanto existiam leitores ativos que foram colocados para dormir propositalmente, levando ao atraso da liberação do lock. Após a finalização das leituras, o escritor retomou a execução e realizou a escrita no buffer.

Esse teste demonstrou que a lógica de meta-locking implementada no `BUFFER_NAVEGACAO` impede leituras durante uma escrita ativa e impede escritas enquanto há leitores ativos. Dessa forma, foi validada a exclusão adequada entre leitores e escritores, reduzindo o risco de inconsistências no acesso ao buffer compartilhado.

3.8 Teste de Finalização das Threads

Outro aspecto avaliado foi a finalização controlada das threads. Durante os testes iniciais, observou-se que algumas threads poderiam permanecer bloqueadas indefinidamente caso ficassem aguardando eventos que não seriam mais produzidos, causando deadlock. Esse comportamento foi corrigido por meio da inclusão de flags de finalização e notificações adicionais ao término das tarefas produtoras.

Nos testes finais, a função principal conseguiu aguardar a finalização de todas as threads por meio de `join()`. Foram registradas mensagens indicando a conclusão das threads de distância percorrida, inspeção por câmera, coletor de dados, reconstrução do teto e controle de navegação, seguida da impressão dos valores escritos em cada buffer e da finalização do sistema.

A thread de controle de navegação permaneceu ativa por mais tempo devido a atrasos artificiais inseridos para simular um consumidor mais lento, mas finalizou corretamente ao término de suas iterações. Dessa forma, foi validado que o sistema encerra de maneira controlada, sem a necessidade de interrupção manual, já que todas as tarefas possuem uma condição de término definida.

3.9 Considerações Finais dos Testes

De forma geral, os testes preliminares indicaram que a implementação atual atende aos objetivos definidos para a primeira etapa do projeto. Foram validadas a criação das tarefas concorrentes, a comunicação por buffers, a sincronização por mutexes e variáveis de condição, o bloqueio correto em situações de buffer cheio e vazio, o controle de leitores e escritores no buffer de navegação, o acionamento da câmera por evento simulado e a finalização controlada das threads.

Apesar disso, os testes ainda foram realizados com dados simulados e em ambiente controlado. A validação completa do sistema dependerá da etapa seguinte.

Todos os prints e testes realizados encontram-se no apêndice desse relatório.

4 Conclusão

Na primeira etapa do trabalho foi desenvolvida a arquitetura inicial do sistema de inspeção automática de túneis, contemplando a implementação das principais tarefas concorrentes responsáveis pela navegação, reconstrução do teto, coleta de dados e inspeção por câmera. O sistema foi estruturado utilizando processos, múltiplas threads e mecanismos de comunicação e sincronização em C++, com foco na aplicação dos conceitos estudados na disciplina de Automação em Tempo Real.

A comunicação entre processos foi realizada por meio de memória compartilhada, enquanto a troca de dados entre tarefas concorrentes foi organizada utilizando buffers compartilhados protegidos por `mutexes` e variáveis de condição. Para o `BUFFER_NAVEGACAO`, foi implementada uma lógica de leitores e escritores utilizando meta-locking, permitindo múltiplas leituras simultâneas e garantindo exclusão mútua durante operações de escrita. Já o `BUFFER_NIVEL` foi estruturado no modelo produtor-consumidor, controlando corretamente situações de buffer cheio e buffer vazio.

Além da implementação da arquitetura básica do sistema, foram realizados diversos testes preliminares para validar o comportamento concorrente da aplicação. Os testes demonstraram que as tarefas conseguem trocar informações corretamente, que os mecanismos de sincronização evitam condições de corrida e que os eventos de acionamento da câmera são tratados adequadamente. Também foi validada a finalização controlada das threads e o correto funcionamento dos buffers compartilhados em diferentes cenários de execução.

Os resultados obtidos até o momento indicam que a base estrutural do sistema está funcionando de forma coerente e atende aos objetivos definidos para a primeira etapa do projeto. Apesar disso, parte das funcionalidades ainda utiliza dados simulados e simplificações temporárias, principalmente relacionadas à física da navegação, ao processamento da câmera e à interface de operação remota.

Como próximos passos, pretende-se concluir a integração completa da simulação física do robô e do túnel, implementar a comunicação remota via protocolo MQTT e desenvolver as interfaces gráficas de simulação e operação remota. Além disso, serão incorporados testes mais avançados de desempenho, tempo de resposta e escalonabilidade das tarefas, avaliando o comportamento do sistema sob diferentes cargas de processamento e frequências de execução. Também serão realizados testes adicionais de robustez e sincronização para validar o funcionamento do sistema completo em cenários mais próximos de uma aplicação real.

5 Apêndice

5.1 Testes realizados

```
● aninhamelado@MacBook-Air-de-Ana-4 Trabalho-de-ATR % make run
./programa
[19:12:47] [PROCESSO] Processo comando_navegacao criado
Comando de navegação escreveu na memória compartilhada:
c_automatico = 1
j_sp_velocidade = 50
Controle de navegação lendo memória compartilhada:
c_automatico = 1
j_sp_velocidade = 50
```

Figura 3: Execução inicial do programa

```
[19:12:47] [MAIN] Buffers inicializados
[19:12:47] [MAIN] Criando thread distancia_percorrida
[19:12:47] [MAIN] Criando thread inspecao_camera
[19:12:47] [MAIN] Criando thread coletor_dados
[19:12:47] [DISTANCIA] Escritor demora para segurar acesso ao BUFFER_NAVEGACAO
[19:12:47] [MAIN] Criando thread reconstrucao_teto
[19:12:47] [COLETOR] Thread inicializada
```

Figura 4: Criação das Threads

```
TESTE SEM DETECÇÃO DE FALHAS

aninhamelado@MacBook-Air-de-Ana-4 Trabalho-de-ATR % make run
./programa
[17:51:45] [PROCESSO] Processo comando_navegacao criado
Comando de navegação escreveu na memória compartilhada:
c_automatico = 1
j_sp_velocidade = 50
Controle de navegação lendo memória compartilhada:
c_automatico = 1
j_sp_velocidade = 50
[17:51:45] [MAIN] Buffers inicializados
[17:51:45] [MAIN] Criando thread distancia_percorrida
[17:51:45] [MAIN] Criando thread inspecao_camera
[17:51:45] [MAIN] Criando thread coletor_dados
[17:51:45] [MAIN] Criando thread reconstrucao_teto
[17:51:45] [MAIN] Criando thread controle_navegacao
[17:51:45] [MAIN] Esperando thread 0
[17:51:45] [COLETOR] Thread inicializada
[17:51:45] [RECONSTRUCAO] Thread inicializada
[17:51:45] [COLETOR] Buffer vazio -> aguardando dados
[17:51:45] [CONTROLE] Posição lida (navegação): 0.000000
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 10.782785
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 11.484781
```

[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 26.471970
[17:51:45] [COLETOR] Posição lida (nível): 10.782785
[17:51:45] [COLETOR] Posição lida (nível): 11.484781
[17:51:45] [COLETOR] Posição lida (nível): 26.471970
[17:51:45] [COLETOR] Buffer vazio -> aguardando dados
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 39.097961
[17:51:45] [COLETOR] Posição lida (nível): 39.097961
[17:51:45] [COLETOR] Buffer vazio -> aguardando dados
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 68.813644
[17:51:45] [COLETOR] Posição lida (nível): 68.813644
[17:51:45] [COLETOR] Buffer vazio -> aguardando dados
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 64.550240
[17:51:45] [COLETOR] Posição lida (nível): 64.550240
[17:51:45] [COLETOR] Buffer vazio -> aguardando dados
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 58.606701
[17:51:45] [COLETOR] Posição lida (nível): 58.606701
[17:51:45] [COLETOR] Buffer vazio -> aguardando dados
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 59.807774
[17:51:45] [COLETOR] Posição lida (nível): 59.807774
[17:51:45] [COLETOR] Buffer vazio -> aguardando dados
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 63.561176
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 41.966251
[17:51:45] [COLETOR] Posição lida (nível): 63.561176
[17:51:45] [COLETOR] Posição lida (nível): 41.966251
[17:51:45] [COLETOR] Buffer vazio -> aguardando dados
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 53.831238
[17:51:45] [COLETOR] Posição lida (nível): 53.831238
[17:51:45] [COLETOR] Buffer vazio -> aguardando dados
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 69.043571
[17:51:45] [COLETOR] Posição lida (nível): 69.043571
[17:51:45] [COLETOR] Buffer vazio -> aguardando dados
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 32.942486
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 31.726030
[17:51:45] [COLETOR] Posição lida (nível): 32.942486
[17:51:45] [COLETOR] Posição lida (nível): 31.726030
[17:51:45] [COLETOR] Buffer vazio -> aguardando dados
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 96.737633
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 42.435276
[17:51:45] [COLETOR] Posição lida (nível): 96.737633
[17:51:45] [COLETOR] Posição lida (nível): 42.435276
[17:51:45] [COLETOR] Buffer vazio -> aguardando dados
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 53.318428
[17:51:45] [COLETOR] Posição lida (nível): 53.318428
[17:51:45] [COLETOR] Buffer vazio -> aguardando dados
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 43.819653
[17:51:45] [COLETOR] Posição lida (nível): 43.819653
[17:51:45] [COLETOR] Buffer vazio -> aguardando dados
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 86.956482
[17:51:45] [COLETOR] Posição lida (nível): 86.956482
[17:51:45] [COLETOR] Buffer vazio -> aguardando dados
[17:51:45] [RECONSTRUCAO] Posição escrita (nível): 38.257179

```

[17:51:45] [RECONSTRUCAO] Thread finalizada
[17:51:45] [COLETOR] Posição lida (nível): 38.257179
[17:51:46] [DISTANCIA] Posição escrita (navegação): 28.958828
[17:51:47] [CONTROLE] Posição lida (navegação): 0.000000
[17:51:47] [DISTANCIA] Posição escrita (navegação): 93.278160
[17:51:48] [DISTANCIA] Posição escrita (navegação): 5.549335
[17:51:49] [CONTROLE] Posição lida (navegação): 5.549335
[17:51:49] [DISTANCIA] Posição escrita (navegação): 59.613941
[17:51:50] [DISTANCIA] Posição escrita (navegação): 5.053784
[17:51:51] [CONTROLE] Posição lida (navegação): 59.613941
[17:51:51] [DISTANCIA] Posição escrita (navegação): 96.715065
[17:51:52] [DISTANCIA] Posição escrita (navegação): 24.145536
[17:51:53] [CONTROLE] Posição lida (navegação): 5.053784
[17:51:53] [DISTANCIA] Posição escrita (navegação): 98.272110
[17:51:54] [DISTANCIA] Posição escrita (navegação): 64.581703
[17:51:55] [CONTROLE] Posição lida (navegação): 96.715065
[17:51:56] [DISTANCIA] Posição escrita (navegação): 60.326897
[17:51:57] [DISTANCIA] Posição escrita (navegação): 94.607086
[17:51:57] [CONTROLE] Posição lida (navegação): 24.145536
[17:51:58] [DISTANCIA] Posição escrita (navegação): 63.498756
[17:51:59] [DISTANCIA] Posição escrita (navegação): 72.532227
[17:51:59] [CONTROLE] Posição lida (navegação): 98.272110
[17:52:00] [DISTANCIA] Posição escrita (navegação): 8.689487
[17:52:01] [DISTANCIA] Posição escrita (navegação): 33.802132
[17:52:01] [CONTROLE] Posição lida (navegação): 64.581703
[17:52:02] [DISTANCIA] Posição escrita (navegação): 65.608650
[17:52:03] [DISTANCIA] Posição escrita (navegação): 84.797623
[17:52:03] [CONTROLE] Posição lida (navegação): 60.326897
[17:52:04] [DISTANCIA] Posição escrita (navegação): 97.202446
[17:52:05] [DISTANCIA] Posição escrita (navegação): 57.833843
[17:52:06] [CONTROLE] Posição lida (navegação): 94.607086
[17:52:06] [DISTANCIA] Posição escrita (navegação): 1.694813
[17:52:06] [MAIN] Thread 0 finalizada
[17:52:06] [MAIN] Esperando thread 1
[17:52:06] [MAIN] Thread 1 finalizada
[17:52:06] [MAIN] Esperando thread 2
[17:52:06] [MAIN] Thread 2 finalizada
[17:52:06] [MAIN] Esperando thread 3
[17:52:06] [MAIN] Thread 3 finalizada
[17:52:06] [MAIN] Esperando thread 4
[17:52:08] [CONTROLE] Posição lida (navegação): 63.498756
[17:52:10] [CONTROLE] Posição lida (navegação): 72.532227
[17:52:12] [CONTROLE] Posição lida (navegação): 8.689487
[17:52:14] [CONTROLE] Posição lida (navegação): 33.802132
[17:52:16] [CONTROLE] Posição lida (navegação): 65.608650
[17:52:18] [CONTROLE] Posição lida (navegação): 84.797623
[17:52:20] [CONTROLE] Posição lida (navegação): 97.202446
[17:52:22] [CONTROLE] Posição lida (navegação): 57.833843
[17:52:24] [CONTROLE] Posição lida (navegação): 1.694813
[17:52:26] [MAIN] Thread 4 finalizada

```

```
Buffer navegação: 94.6071 63.4988 72.5322 8.68949 33.8021 65.6087 84.7976
↳ 97.2024 57.8338 1.69481
Buffer nível: 53.8312 69.0436 32.9425 31.726 96.7376 42.4353 53.3184 43.8197
↳ 86.9565 38.2572
[17:52:26] [MAIN] Execução finalizada
```

TESTE COM UMA FALHA DETECTADA

```
aninhamelado@MacBook-Air-de-Ana-4 Trabalho-de-ATR % make run
./programa
[18:15:56] [PROCESSO] Processo comando_navegacao criado
Comando de navegação escreveu na memória compartilhada:
c_automatico = 1
j_sp_velocidade = 50
Controle de navegação lendo memória compartilhada:
c_automatico = 1
j_sp_velocidade = 50
[18:15:56] [MAIN] Buffers inicializados
[18:15:56] [MAIN] Criando thread distancia_percorrida
[18:15:56] [MAIN] Criando thread inspecao_camera
[18:15:56] [MAIN] Criando thread coletor_dados
[18:15:56] [MAIN] Criando thread reconstrucao_teto
[18:15:56] [COLETOR] Thread inicializada
[18:15:56] [RECONSTRUCAO] Thread inicializada
[18:15:56] [COLETOR] Buffer vazio -> aguardando dados
[18:15:56] [MAIN] Criando thread controle_navegacao
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 74.495483
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 42.898727
[18:15:56] [CONTROLE] Posição lida (navegação): 0.000000
[18:15:56] [MAIN] Esperando thread 0
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 34.575493
[18:15:56] [COLETOR] Posição lida (nível): 74.495483
[18:15:56] [COLETOR] Posição lida (nível): 42.898727
[18:15:56] [COLETOR] Posição lida (nível): 34.575493
[18:15:56] [COLETOR] Buffer vazio -> aguardando dados
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 98.192505
[18:15:56] [COLETOR] Posição lida (nível): 98.192505
[18:15:56] [COLETOR] Buffer vazio -> aguardando dados
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 15.192101
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 73.814850
[18:15:56] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↳ reduzida
[18:15:56] [COLETOR] Posição lida (nível): 15.192101
[18:15:56] [COLETOR] Posição lida (nível): 73.814850
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 97.350990
[18:15:56] [CAMERA] Inspeção iniciada
[18:15:56] [CAMERA] Inspeção finalizada
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 55.099091
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 48.265957
[18:15:56] [COLETOR] Posição lida (nível): 97.350990
[18:15:56] [COLETOR] Posição lida (nível): 55.099091
```

```

[18:15:56] [COLETOR] Posição lida (nível): 48.265957
[18:15:56] [COLETOR] Buffer vazio -> aguardando dados
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 30.761057
[18:15:56] [COLETOR] Posição lida (nível): 30.761057
[18:15:56] [COLETOR] Buffer vazio -> aguardando dados
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 49.804440
[18:15:56] [COLETOR] Posição lida (nível): 49.804440
[18:15:56] [COLETOR] Buffer vazio -> aguardando dados
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 46.795132
[18:15:56] [COLETOR] Posição lida (nível): 46.795132
[18:15:56] [COLETOR] Buffer vazio -> aguardando dados
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 12.473819
[18:15:56] [COLETOR] Posição lida (nível): 12.473819
[18:15:56] [COLETOR] Buffer vazio -> aguardando dados
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 68.368179
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 34.957470
[18:15:56] [COLETOR] Posição lida (nível): 68.368179
[18:15:56] [COLETOR] Posição lida (nível): 34.957470
[18:15:56] [COLETOR] Buffer vazio -> aguardando dados
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 90.110184
[18:15:56] [COLETOR] Posição lida (nível): 90.110184
[18:15:56] [COLETOR] Buffer vazio -> aguardando dados
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 39.241226
[18:15:56] [COLETOR] Posição lida (nível): 39.241226
[18:15:56] [COLETOR] Buffer vazio -> aguardando dados
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 31.790064
[18:15:56] [COLETOR] Posição lida (nível): 31.790064
[18:15:56] [COLETOR] Buffer vazio -> aguardando dados
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 93.427567
[18:15:56] [RECONSTRUCAO] Posição escrita (nível): 61.848675
[18:15:56] [RECONSTRUCAO] Thread finalizada
[18:15:56] [COLETOR] Posição lida (nível): 93.427567
[18:15:56] [COLETOR] Posição lida (nível): 61.848675
[18:15:57] [DISTANCIA] Posição escrita (navegação): 62.380260
[18:15:58] [CONTROLE] Posição lida (navegação): 0.000000
[18:15:58] [DISTANCIA] Posição escrita (navegação): 51.585777
[18:15:59] [DISTANCIA] Posição escrita (navegação): 62.972610
[18:16:00] [CONTROLE] Posição lida (navegação): 62.972610
[18:16:00] [DISTANCIA] Posição escrita (navegação): 65.160225
[18:16:01] [DISTANCIA] Posição escrita (navegação): 31.197983
[18:16:02] [CONTROLE] Posição lida (navegação): 65.160225
[18:16:02] [DISTANCIA] Posição escrita (navegação): 45.259418
[18:16:03] [DISTANCIA] Posição escrita (navegação): 81.480118
[18:16:04] [CONTROLE] Posição lida (navegação): 31.197983
[18:16:04] [DISTANCIA] Posição escrita (navegação): 19.269875
[18:16:05] [DISTANCIA] Posição escrita (navegação): 17.381445
[18:16:06] [CONTROLE] Posição lida (navegação): 45.259418
[18:16:06] [DISTANCIA] Posição escrita (navegação): 43.342255
[18:16:07] [DISTANCIA] Posição escrita (navegação): 1.306038
[18:16:08] [CONTROLE] Posição lida (navegação): 81.480118
[18:16:08] [DISTANCIA] Posição escrita (navegação): 12.224937

```

```

[18:16:09] [DISTANCIA] Posição escrita (navegação): 75.290260
[18:16:10] [CONTROLE] Posição lida (navegação): 19.269875
[18:16:10] [DISTANCIA] Posição escrita (navegação): 55.725075
[18:16:11] [DISTANCIA] Posição escrita (navegação): 83.067741
[18:16:12] [CONTROLE] Posição lida (navegação): 17.381445
[18:16:12] [DISTANCIA] Posição escrita (navegação): 20.277987
[18:16:13] [DISTANCIA] Posição escrita (navegação): 60.572338
[18:16:14] [CONTROLE] Posição lida (navegação): 43.342255
[18:16:14] [DISTANCIA] Posição escrita (navegação): 44.572971
[18:16:15] [DISTANCIA] Posição escrita (navegação): 38.374176
[18:16:16] [CONTROLE] Posição lida (navegação): 1.306038
[18:16:16] [DISTANCIA] Posição escrita (navegação): 34.950138
[18:16:16] [MAIN] Thread 0 finalizada
[18:16:16] [MAIN] Esperando thread 1
[18:16:16] [MAIN] Thread 1 finalizada
[18:16:16] [MAIN] Esperando thread 2
[18:16:16] [MAIN] Thread 2 finalizada
[18:16:16] [MAIN] Esperando thread 3
[18:16:16] [MAIN] Thread 3 finalizada
[18:16:16] [MAIN] Esperando thread 4
[18:16:18] [CONTROLE] Posição lida (navegação): 12.224937
[18:16:20] [CONTROLE] Posição lida (navegação): 75.290260
[18:16:22] [CONTROLE] Posição lida (navegação): 55.725075
[18:16:24] [CONTROLE] Posição lida (navegação): 83.067741
[18:16:26] [CONTROLE] Posição lida (navegação): 20.277987
[18:16:28] [CONTROLE] Posição lida (navegação): 60.572338
[18:16:30] [CONTROLE] Posição lida (navegação): 44.572971
[18:16:32] [CONTROLE] Posição lida (navegação): 38.374176
[18:16:34] [CONTROLE] Posição lida (navegação): 34.950138
[18:16:36] [MAIN] Thread 4 finalizada
Buffer navegação: 1.30604 12.2249 75.2903 55.7251 83.0677 20.278 60.5723
↪ 44.573 38.3742 34.9501
Buffer nível: 49.8044 46.7951 12.4738 68.3682 34.9575 90.1102 39.2412 31.7901
↪ 93.4276 61.8487
[18:16:36] [MAIN] Execução finalizada

```

TESTE COM MÚLTIPLAS FALHAS

```

aninhamelado@MacBook-Air-de-Ana-4 Trabalho-de-ATR % make run
./programa
[18:14:34] [PROCESS0] Processo comando_navegacao criado
Comando de navegação escreveu na memória compartilhada:
c_automatico = 1
j_sp_velocidade = 50
Controle de navegação lendo memória compartilhada:
c_automatico = 1
j_sp_velocidade = 50
[18:14:34] [MAIN] Buffers inicializados
[18:14:34] [MAIN] Criando thread distancia_percorrida
[18:14:34] [MAIN] Criando thread inspecao_camera
[18:14:34] [MAIN] Criando thread coletor_dados

```

```

[18:14:34] [MAIN] Criando thread reconstrucao_teto
[18:14:34] [COLETOR] Thread inicializada
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [RECONSTRUCAO] Thread inicializada
[18:14:34] [MAIN] Criando thread controle_navegacao
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 11.893382
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↳ reduzida
[18:14:34] [MAIN] Esperando thread 0
[18:14:34] [COLETOR] Posição lida (nível): 11.893382
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [CONTROLE] Posição lida (navegação): 0.000000
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 33.125107
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↳ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 33.125107
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 70.220352
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↳ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 70.220352
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 38.717987
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↳ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 38.717987
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 89.701645
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↳ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 89.701645
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 17.111982
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↳ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 17.111982
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 85.353203

```

[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
 ↳ reduzida
 [18:14:34] [COLETOR] Posição lida (nível): 85.353203
 [18:14:34] [COLETOR] Buffer vazio -> aguardando dados
 [18:14:34] [RECONSTRUCAO] Posição escrita (nível): 42.381588
 [18:14:34] [CAMERA] Inspeção iniciada
 [18:14:34] [CAMERA] Inspeção finalizada
 [18:14:34] [COLETOR] Posição lida (nível): 42.381588
 [18:14:34] [COLETOR] Buffer vazio -> aguardando dados
 [18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
 ↳ reduzida
 [18:14:34] [CAMERA] Inspeção iniciada
 [18:14:34] [CAMERA] Inspeção finalizada
 [18:14:34] [RECONSTRUCAO] Posição escrita (nível): 18.459126
 [18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
 ↳ reduzida
 [18:14:34] [COLETOR] Posição lida (nível): 18.459126
 [18:14:34] [COLETOR] Buffer vazio -> aguardando dados
 [18:14:34] [CAMERA] Inspeção iniciada
 [18:14:34] [CAMERA] Inspeção finalizada
 [18:14:34] [RECONSTRUCAO] Posição escrita (nível): 78.904755
 [18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
 ↳ reduzida
 [18:14:34] [COLETOR] Posição lida (nível): 78.904755
 [18:14:34] [CAMERA] Inspeção iniciada
 [18:14:34] [CAMERA] Inspeção finalizada
 [18:14:34] [COLETOR] Buffer vazio -> aguardando dados
 [18:14:34] [RECONSTRUCAO] Posição escrita (nível): 46.468765
 [18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
 ↳ reduzida
 [18:14:34] [COLETOR] Posição lida (nível): 46.468765
 [18:14:34] [COLETOR] Buffer vazio -> aguardando dados
 [18:14:34] [CAMERA] Inspeção iniciada
 [18:14:34] [CAMERA] Inspeção finalizada
 [18:14:34] [RECONSTRUCAO] Posição escrita (nível): 68.231094
 [18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
 ↳ reduzida
 [18:14:34] [COLETOR] Posição lida (nível): 68.231094
 [18:14:34] [CAMERA] Inspeção iniciada
 [18:14:34] [CAMERA] Inspeção finalizada
 [18:14:34] [COLETOR] Buffer vazio -> aguardando dados
 [18:14:34] [RECONSTRUCAO] Posição escrita (nível): 70.282280
 [18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
 ↳ reduzida
 [18:14:34] [COLETOR] Posição lida (nível): 70.282280
 [18:14:34] [COLETOR] Buffer vazio -> aguardando dados
 [18:14:34] [CAMERA] Inspeção iniciada
 [18:14:34] [CAMERA] Inspeção finalizada
 [18:14:34] [RECONSTRUCAO] Posição escrita (nível): 47.991817
 [18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
 ↳ reduzida


```

[18:14:34] [COLETOR] Posição lida (nível): 47.991817
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 55.565323
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↪ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 55.565323
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 76.516632
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↪ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 76.516632
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 59.348728
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↪ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 59.348728
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 8.146796
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↪ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 8.146796
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 68.844254
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↪ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 68.844254
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:34] [COLETOR] Buffer vazio -> aguardando dados
[18:14:34] [RECONSTRUCAO] Posição escrita (nível): 26.924643
[18:14:34] [RECONSTRUCAO] Falha detectada -> câmera acionada e velocidade
↪ reduzida
[18:14:34] [COLETOR] Posição lida (nível): 26.924643
[18:14:34] [RECONSTRUCAO] Thread finalizada
[18:14:34] [CAMERA] Inspeção iniciada
[18:14:34] [CAMERA] Inspeção finalizada
[18:14:35] [DISTANCIA] Posição escrita (navegação): 93.736534
[18:14:36] [DISTANCIA] Escritor aguardando acesso ao buffer navegacao
[18:14:36] [CONTROLE] Posição lida (navegação): 0.000000
[18:14:36] [DISTANCIA] Escritor retomou execução
[18:14:36] [DISTANCIA] Posição escrita (navegação): 80.958572

```

```

[18:14:37] [DISTANCIA] Posição escrita (navegação): 35.029560
[18:14:38] [CONTROLE] Posição lida (navegação): 35.029560
[18:14:38] [DISTANCIA] Posição escrita (navegação): 91.344322
[18:14:39] [DISTANCIA] Posição escrita (navegação): 96.001343
[18:14:40] [CONTROLE] Posição lida (navegação): 91.344322
[18:14:40] [DISTANCIA] Posição escrita (navegação): 11.806067
[18:14:41] [DISTANCIA] Posição escrita (navegação): 73.489479
[18:14:42] [CONTROLE] Posição lida (navegação): 96.001343
[18:14:42] [DISTANCIA] Posição escrita (navegação): 10.448226
[18:14:43] [DISTANCIA] Posição escrita (navegação): 62.412590
[18:14:44] [CONTROLE] Posição lida (navegação): 11.806067
[18:14:44] [DISTANCIA] Posição escrita (navegação): 64.524155
[18:14:45] [DISTANCIA] Posição escrita (navegação): 19.370922
[18:14:46] [CONTROLE] Posição lida (navegação): 73.489479
[18:14:46] [DISTANCIA] Posição escrita (navegação): 46.210743
[18:14:47] [DISTANCIA] Posição escrita (navegação): 97.758255
[18:14:48] [CONTROLE] Posição lida (navegação): 10.448226
[18:14:48] [DISTANCIA] Posição escrita (navegação): 80.271263
[18:14:49] [DISTANCIA] Posição escrita (navegação): 34.450363
[18:14:50] [CONTROLE] Posição lida (navegação): 62.412590
[18:14:50] [DISTANCIA] Posição escrita (navegação): 30.559107
[18:14:51] [DISTANCIA] Posição escrita (navegação): 68.277695
[18:14:52] [CONTROLE] Posição lida (navegação): 64.524155
[18:14:52] [DISTANCIA] Posição escrita (navegação): 99.241768
[18:14:53] [DISTANCIA] Posição escrita (navegação): 39.216423
[18:14:54] [CONTROLE] Posição lida (navegação): 19.370922
[18:14:54] [DISTANCIA] Posição escrita (navegação): 28.282803
[18:14:54] [MAIN] Thread 0 finalizada
[18:14:54] [MAIN] Esperando thread 1
[18:14:54] [MAIN] Thread 1 finalizada
[18:14:54] [MAIN] Esperando thread 2
[18:14:54] [MAIN] Thread 2 finalizada
[18:14:54] [MAIN] Esperando thread 3
[18:14:54] [MAIN] Thread 3 finalizada
[18:14:54] [MAIN] Esperando thread 4
[18:14:56] [CONTROLE] Posição lida (navegação): 46.210743
[18:14:58] [CONTROLE] Posição lida (navegação): 97.758255
[18:15:00] [CONTROLE] Posição lida (navegação): 80.271263
[18:15:02] [CONTROLE] Posição lida (navegação): 34.450363
[18:15:04] [CONTROLE] Posição lida (navegação): 30.559107
[18:15:06] [CONTROLE] Posição lida (navegação): 68.277695
[18:15:08] [CONTROLE] Posição lida (navegação): 99.241768
[18:15:10] [CONTROLE] Posição lida (navegação): 39.216423
[18:15:12] [CONTROLE] Posição lida (navegação): 28.282803
[18:15:14] [MAIN] Thread 4 finalizada
Buffer navegação: 19.3709 46.2107 97.7583 80.2713 34.4504 30.5591 68.2777
↪ 99.2418 39.2164 28.2828
Buffer nível: 46.4688 68.2311 70.2823 47.9918 55.5653 76.5166 59.3487 8.1468
↪ 68.8443 26.9246
[18:15:14] [MAIN] Execução finalizada

```

```

[19:06:52] [COLETOR] Buffer de nível vazio -> aguardando dados
[19:06:52] [DISTANCIA] Posição escrita (navegação): 24.064884
[19:06:52] [RECONSTRUCAO] Thread inicializada
[19:06:52] [MAIN] Criando thread controle_navegacao
[19:06:52] [RECONSTRUCAO] Posição escrita (nível): 149.649765 | Ocupação: 1/10
[19:06:52] [DISTANCIA] Posição escrita (navegação): 2.887431
[19:06:52] [CONTROLE] Leitor demora para segurar acesso ao BUFFER_NAVEGACAO
[19:06:52] [RECONSTRUCAO] Posição escrita (nível): 86.107315 | Ocupação: 2/10
[19:06:52] [MAIN] Esperando thread 0
[19:06:52] [COLETOR] Posição lida (nível): 149.649765
[19:06:52] [COLETOR] Posição lida (nível): 86.107315
[19:06:52] [COLETOR] Buffer de nível vazio -> aguardando dados

```

Figura 5: Teste de Buffer de Nível Vazio

```

[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 112.025932 | Ocupação: 1/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 160.867523 | Ocupação: 2/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 122.139771 | Ocupação: 3/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 86.595070 | Ocupação: 4/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 146.985657 | Ocupação: 5/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 39.833908 | Ocupação: 6/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 103.447708 | Ocupação: 7/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 92.925583 | Ocupação: 8/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 125.545197 | Ocupação: 9/10
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 124.471046 | Ocupação: 10/10
[19:13:01] [RECONSTRUCAO] Buffer de nível cheio -> produtor aguardando espaço
[19:13:01] [CONTROLE] Leitor retomou execução
[19:13:01] [DISTANCIA] Posição escrita (navegação): 71.950806
[19:13:01] [DISTANCIA] Escritor aguardando acesso ao buffer navegacao
[19:13:01] [COLETOR] Posição lida (nível): 112.025932
[19:13:01] [COLETOR] Posição lida (nível): 160.867523
[19:13:01] [COLETOR] Posição lida (nível): 122.139771
[19:13:01] [COLETOR] Posição lida (nível): 86.595070
[19:13:01] [COLETOR] Posição lida (nível): 146.985657
[19:13:01] [COLETOR] Posição lida (nível): 39.833908
[19:13:01] [COLETOR] Posição lida (nível): 103.447708
[19:13:01] [COLETOR] Posição lida (nível): 92.925583
[19:13:01] [COLETOR] Posição lida (nível): 125.545197
[19:13:01] [COLETOR] Posição lida (nível): 124.471046
[19:13:01] [COLETOR] Buffer de nível vazio -> aguardando dados
[19:13:01] [RECONSTRUCAO] Espaço do buffer de nível liberado -> produtor retomou execução
[19:13:01] [RECONSTRUCAO] Posição escrita (nível): 52.224766 | Ocupação: 1/10

```

Figura 6: Teste de Buffer de Nível Cheio

```

[19:12:47] [DISTANCIA] Escritor demora para segurar acesso ao BUFFER_NAVEGACAO
[19:12:47] [MAIN] Criando thread reconstrucao_teto
[19:12:47] [COLETOR] Thread inicializada
[19:12:47] [COLETOR] Buffer de nível vazio -> aguardando dados
[19:12:47] [RECONSTRUCAO] Thread inicializada
[19:12:47] [RECONSTRUCAO] Leitor aguardando acesso ao buffer navegacao
[19:12:47] [MAIN] Criando thread controle_navegacao
[19:12:47] [MAIN] Esperando thread 0
[19:12:47] [CONTROLE] Leitor aguardando acesso ao buffer navegacao
[19:12:48] [DISTANCIA] Posição escrita (navegação): 73.074905
[19:12:48] [RECONSTRUCAO] Leitor retomou execução
[19:12:48] [CONTROLE] Leitor retomou execução

```

Figura 7: Teste de Bloqueio de Escritores

```

[19:06:52] [MAIN] Criando thread controle_navegacao
[19:06:52] [RECONSTRUCAO] Posição escrita (nível): 149.649765 | Ocupação: 1/10
[19:06:52] [DISTANCIA] Posição escrita (navegação): 2.887431
[19:06:52] [CONTROLE] Leitor demora para segurar acesso ao BUFFER_NAVEGACAO
[19:06:52] [RECONSTRUCAO] Posição escrita (nível): 86.107315 | Ocupação: 2/10
[19:06:52] [MAIN] Esperando thread 0
[19:06:52] [COLETOR] Posição lida (nível): 149.649765
[19:06:52] [COLETOR] Posição lida (nível): 86.107315
[19:06:52] [COLETOR] Buffer de nível vazio -> aguardando dados
[19:06:52] [DISTANCIA] Escritor aguardando acesso ao buffer navegacao
[19:06:53] [CONTROLE] Posição lida (navegação): 53.040310
[19:06:53] [DISTANCIA] Escritor retomou execução
[19:06:53] [DISTANCIA] Posição escrita (navegação): 99.067177
[19:06:53] [DISTANCIA] Escritor aguardando acesso ao buffer navegacao
[19:06:53] [CONTROLE] Leitor demora para segurar acesso ao BUFFER_NAVEGACAO

```

Figura 8: Teste de Bloqueio de Leitores

```

[19:13:18] [MAIN] Thread 0 finalizada
[19:13:18] [MAIN] Esperando thread 1
[19:13:18] [MAIN] Thread 1 finalizada
[19:13:18] [MAIN] Esperando thread 2
[19:13:18] [MAIN] Thread 2 finalizada
[19:13:18] [MAIN] Esperando thread 3
[19:13:18] [MAIN] Thread 3 finalizada
[19:13:18] [MAIN] Esperando thread 4
[19:13:19] [CONTROLE] Posição lida (navegação): 97.285744
[19:13:20] [CONTROLE] Posição lida (navegação): 12.477365
[19:13:21] [CONTROLE] Posição lida (navegação): 46.973194
[19:13:22] [CONTROLE] Posição lida (navegação): 20.479900
[19:13:23] [CONTROLE] Posição lida (navegação): 82.494011
[19:13:24] [CONTROLE] Posição lida (navegação): 25.792789
[19:13:24] [MAIN] Thread 4 finalizada
Buffer navegacao: 14.1857 37.1402 80.635 85.4882 97.2857 12.4774 46.9732 20.4799 82.494 25.7928
Buffer nível: 146.986 39.8339 103.448 92.9256 125.545 124.471 52.2248 108.817 163.728 65.7006
[19:13:24] [MAIN] Execução finalizada

```

Figura 9: Teste de Finalização das Threads

5.2 includes.hpp

```

1  #pragma once
2
3  #include <iostream>
4  #include <vector>
5  #include <chrono>
6  #include <cstdlib>
7  #include <cstring>
8  #include <csignal>
9  #include <random>
10
11 #include <thread>
12 #include <atomic>
13 #include <mutex>
14 #include <condition_variable>
15 #include <semaphore>
16 #include <pthread.h>
17 #include <iomanip>

```

```

18
19 #include <unistd.h>
20 #include <sys/ipc.h>
21 #include <sys/shm.h>
22 #include <sys/wait.h>

```

5.3 main.cpp

```

1  #include "include/includes.hpp"
2  #include "include/shared_memory.hpp"
3  #include "include/sincronizacao.hpp"
4  #include "include/simulacao.hpp"
5
6  int main (){
7
8      simulacao();
9
10     // Generate a unique key for the shared memory segment
11     key_t key = IPC_PRIVATE; // Use IPC_PRIVATE for a unique key
12
13     // Create a shared memory segment
14     int shmid = shmget(key, sizeof(MemoriaCompartilhada), 0666 | IPC_CREAT);
15
16     if (shmid < 0) {
17         perror("Erro ao criar memória compartilhada");
18         return 1;
19     }
20
21     // Attach to the shared memory
22     MemoriaCompartilhada* shm = (MemoriaCompartilhada*) shmat(shmid, nullptr,
23         ↪ 0);
24
25     if (shm == (void*) -1) {
26         perror("Erro ao anexar memória compartilhada");
27         return 1;
28     }
29
30     // Inicializa os valores da memória compartilhada
31     shm->i_encoder = false;
32     shm->i_lidar = 0;
33
34     shm->o_liga_camera = false;
35     shm->o_aceleracao = 0;
36
37     shm->e_inspecao = false;
38     shm->e_automatico = false;
39
40     shm->c_automatico = false;
41     shm->c_man = false;
42     shm->j_sp_velocidade = 0;
43
44 }

```

```

43     //INICIALIZAÇÃO PROCESSOS
44
45     pid_t pid;
46
47     pid = fork();
48
49     if (pid == 0){
50
51         log_message(
52             "PROCESSO",
53             "Processo comando_navegacao criado"
54         );
55
56         comando_navegacao ();
57
58         // Exemplo: filho escreve comando remoto
59         shm->c_automatico = true;
60         shm->j_sp_velocidade = 50;
61
62         std::cout << "Comando de navegação escreveu na memória compartilhada:"
63             << std::endl;
64         std::cout << "c_automatico = " << shm->c_automatico << std::endl;
65         std::cout << "j_sp_velocidade = " << shm->j_sp_velocidade << std::endl;
66
67         // Desanexa memória no filho
68         shmdt(shm);
69     }
70
71     else if (pid < 0){
72         perror ("Erro ao criar processo\n");
73         exit(1);
74     }
75
76     else {
77
78         wait(nullptr);
79
80         std::cout << "Controle de navegação lendo memória compartilhada:" <<
81             << std::endl;
82         std::cout << "c_automatico = " << shm->c_automatico << std::endl;
83         std::cout << "j_sp_velocidade = " << shm->j_sp_velocidade << std::endl;
84
85         //INICIALIZAÇÃO DE BUFFERS
86         std::vector <float> BUFFER_NAVEGACAO (ELEMENTOS_BUFFERS); //posição do
87             << carrinho
88         std::vector <float> BUFFER_NIVEL (ELEMENTOS_BUFFERS); //leitura de
89             << nivel
90
91         log_message("MAIN","Buffers inicializados");
92
93         //INICIALIZAÇÃO AS THREADS

```

```

90
91     std::mutex mutex_navegacao;
92     std::mutex mutex_nivel;
93     std::mutex mutex_camera;
94
95     std::vector <std::thread> threads_navegacao;
96
97     log_message(
98         "MAIN",
99         "Criando thread distancia_percorrida"
100    );
101
102    threads_navegacao.emplace_back(
103        distancia_percorrida,
104        std::ref(mutex_navegacao),
105        std::ref(BUFFER_NAVEGACAO)
106    );
107
108    log_message(
109        "MAIN",
110        "Criando thread inspecao_camera"
111    );
112
113    threads_navegacao.emplace_back(
114        inspecao_camera,
115        std::ref(mutex_camera),
116        shm
117    );
118
119    log_message(
120        "MAIN",
121        "Criando thread coletor_dados"
122    );
123
124    threads_navegacao.emplace_back(
125        coletor_dados,
126        std::ref(mutex_nivel),
127        std::ref(BUFFER_NIVEL)
128    );
129
130    log_message(
131        "MAIN",
132        "Criando thread reconstrucao_teto"
133    );
134
135    threads_navegacao.emplace_back(
136        reconstrucao_teto,
137        std::ref(mutex_navegacao),
138        std::ref(mutex_nivel),
139        std::ref(mutex_camera),
140        std::ref(BUFFER_NAVEGACAO),

```

```

141         std::ref(BUFFER_NIVEL),
142         shm
143     );
144
145     log_message(
146         "MAIN",
147         "Criando thread controle_navegacao"
148     );
149
150     threads_navegacao.emplace_back(
151         controle_navegacao,
152         std::ref(mutex_navegacao),
153         std::ref(BUFFER_NAVEGACAO)
154     );
155
156     for (size_t i = 0; i < threads_navegacao.size(); i++) {
157         log_message("MAIN", "Esperando thread " + std::to_string(i));
158
159         if (threads_navegacao[i].joinable()) {
160             threads_navegacao[i].join();
161         }
162
163         log_message("MAIN", "Thread " + std::to_string(i) + " finalizada");
164     }
165
166     std::cout << "Buffer navegação: ";
167
168     for (auto i : BUFFER_NAVEGACAO){
169         std::cout << i << " ";
170     }
171
172     std::cout << std::endl;
173
174     std::cout << "Buffer nível: ";
175
176     for (auto i : BUFFER_NIVEL){
177         std::cout << i << " ";
178     }
179
180     std::cout << std::endl;
181
182     log_message(
183         "MAIN",
184         "Execução finalizada"
185     );
186 }
187
188 // Desanexa memória no pai
189 shmdt(shm);
190
191 // Remove segmento de memória compartilhada

```



```

192     shmctl(shmid, IPC_RMID, nullptr);
193
194     return 0;
195 }

```

5.4 sincronizacao.cpp

```

1  #include "../include/sincronizacao.hpp"
2
3  #define ELEMENTOS_BUFFERS 10
4  const int SHM_SIZE = 1024; // Size of the shared memory segment
5
6  //sincronização
7  std::condition_variable camera;
8
9  //teste para finalizar a camera
10 int eventos_camera = 0;
11 bool finalizar_camera = false;
12
13 //Variaveis de condição buffers
14 std::condition_variable leitura_buffer_navegacao;
15 std::condition_variable escrita_buffer_navegacao;
16
17 int AR_NAVEGACAO = 0;
18 int WR_NAVEGACAO = 0;
19 int AW_NAVEGACAO = 0;
20 int WW_NAVEGACAO = 0;
21
22 std::condition_variable leitura_buffer_nivel;
23 std::condition_variable escrita_buffer_nivel;
24 int dados_nivel = 0;
25
26 //Funções auxiliares de debug
27 std::mutex mutex_log;
28
29 void log_message(const std::string& thread,
30                 const std::string& mensagem){
31
32     std::lock_guard<std::mutex> lock(mutex_log);
33
34     auto agora = std::chrono::system_clock::now();
35
36     auto tempo = std::chrono::system_clock::to_time_t(agora);
37
38     std::cout
39         << "[" << std::put_time(std::localtime(&tempo), "%H:%M:%S") << "]" "
40         << "[" << thread << "]" "
41         << mensagem
42         << std::endl;
43 }
44

```

```

45 float numero_aleatorio_debugg(){
46
47     std::random_device rd;
48     std::mt19937 gen(rd());
49     std::uniform_real_distribution<float> dis(1.0f, 100.0f);
50
51     return dis(gen);
52 }
53
54 //FUNÇÕES DO SISTEMA
55
56 void comando_navegacao(){
57
58 }
59
60 void controle_navegacao(std::mutex &mtx, std::vector <float> &BUFFER){
61
62     int idx = -1; // -1 para corrigir o inicio de leitura do vetor
63
64     for (int i = 0; i<20; i++){
65
66         idx++;
67         idx = idx%ELEMENTOS_BUFFERS;
68
69         std::unique_lock<std::mutex> lock (mtx);
70
71         while((AW_NAVEGACAO + WW_NAVEGACAO) > 0){
72
73             WR_NAVEGACAO++;
74
75             leitura_buffer_navegacao.wait(lock);
76
77             WR_NAVEGACAO--;
78         }
79
80         AR_NAVEGACAO++;
81
82         lock.unlock();
83
84         //SEÇÃO CRÍTICA
85         float leitura = BUFFER[idx];
86         //SEÇÃO CRÍTICA
87
88         lock.lock();
89
90         AR_NAVEGACAO--;
91
92         if(AR_NAVEGACAO == 0 && WW_NAVEGACAO > 0){
93             escrita_buffer_navegacao.notify_one();
94         }
95

```

```

96         lock.unlock();
97
98         log_message(
99             "CONTROLE",
100             "Posição lida (navegação): " + std::to_string(leitura)
101         );
102
103         // TESTE DE BUFFER CHEIO
104         // Consumidor lento para encher o buffer
105         std::this_thread::sleep_for(
106             std::chrono::seconds(2)
107         );
108
109     }
110
111 }
112
113 void distancia_percorrida(std::mutex &mtx, std::vector<float> &BUFFER){
114
115     int idx = -1; // -1 para corrigir o inicio de escrita
116
117     for (int i = 0; i<20; i++){
118
119         idx++;
120         idx = idx % ELEMENTOS_BUFFERS;
121         float escrita = numero_aleatorio_debugg();
122
123         // Produtor lento no início
124         std::this_thread::sleep_for(std::chrono::seconds(1));
125
126         std::unique_lock<std::mutex> lock (mtx);
127
128         //META-LOCKING
129         while((AW_NAVEGACAO + AR_NAVEGACAO) > 0){
130
131             log_message(
132                 "DISTANCIA",
133                 "Escritor aguardando acesso ao buffer navegacao"
134             );
135
136             WW_NAVEGACAO++;
137
138             escrita_buffer_navegacao.wait(lock);
139
140             WW_NAVEGACAO--;
141
142             log_message(
143                 "DISTANCIA",
144                 "Escritor retomou execução"
145             );
146         }

```

```

147
148     AW_NAVEGACAO++;
149
150     lock.unlock();
151
152     //SEÇÃO CRÍTICA
153
154     BUFFER[idx] = escrita;
155
156     //SEÇÃO CRÍTICA
157
158     lock.lock();
159
160     AW_NAVEGACAO--;
161
162     if(WW_NAVEGACAO > 0){
163         escrita_buffer_navegacao.notify_one();
164     }
165
166     else if(WR_NAVEGACAO > 0){
167         leitura_buffer_navegacao.notify_all();
168     }
169
170     lock.unlock();
171
172     log_message(
173         "DISTANCIA",
174         "Posição escrita (navegação): " + std::to_string(escrita)
175     );
176 }
177 }
178
179 void reconstrucao_teto(std::mutex &mtx_navegacao, std::mutex &mtx_nivel,
↪ std::mutex &mtx_camera, std::vector <float> &BUFFER_NAVEGACAO, std::vector
↪ <float> &BUFFER_NIVEL, MemoriaCompartilhada* shm){
180
181     log_message("RECONSTRUCAO", "Thread inicializada");
182
183     int idx_navegacao = -1;
184     int idx_nivel = -1;
185
186     for (int i = 0; i<20; i++){ //No final será trocado para while true
187
188         idx_navegacao++;
189         idx_navegacao = idx_navegacao % ELEMENTOS_BUFFERS;
190
191         std::unique_lock<std::mutex> lock_navegacao(mtx_navegacao);
192
193         while((AW_NAVEGACAO + WW_NAVEGACAO) > 0){
194
195             WR_NAVEGACAO++;

```

```

196         leitura_buffer_navegacao.wait(lock_navegacao);
197
198         WR_NAVEGACAO--;
199     }
200
201     AR_NAVEGACAO++;
202
203     lock_navegacao.unlock();
204
205     //SEÇÃO CRÍTICA NAVEGACAO
206
207     float leitura_navegacao = BUFFER_NAVEGACAO[idx_navegacao];
208
209     //SEÇÃO CRÍTICA NAVEGACAO
210
211     lock_navegacao.lock();
212
213     AR_NAVEGACAO--;
214
215     if(AR_NAVEGACAO == 0 && WW_NAVEGACAO > 0){
216         escrita_buffer_navegacao.notify_one();
217     }
218
219     lock_navegacao.unlock();
220
221     idx_nivel++;
222     idx_nivel = idx_nivel % ELEMENTOS_BUFFERS;
223
224     float escrita = leitura_navegacao + numero_aleatorio_debugg();
225
226     std::unique_lock<std::mutex> lock_nivel (mtx_nivel);
227
228     while(dados_nivel >= 10){
229
230         log_message(
231             "RECONSTRUCAO",
232             "Buffer cheio -> produtor aguardando espaço"
233         );
234
235         escrita_buffer_nivel.wait(lock_nivel);
236     }
237
238     //SEÇÃO CRÍTICA NÍVEL
239     BUFFER_NIVEL[idx_nivel] = escrita;
240     dados_nivel++;
241     leitura_buffer_nivel.notify_one();
242
243     log_message(
244         "RECONSTRUCAO",
245         "Posição escrita (nível): " + std::to_string(escrita)
246

```

```

247     );
248     //SEÇÃO CRÍTICA NÍVEL
249
250     lock_nivel.unlock();
251
252     bool encontrou_falha = true; // teste
253
254     if(encontrou_falha){
255
256         std::lock_guard<std::mutex> lock_camera(mtx_camera);
257
258         shm->e_inspecao = true;
259         shm->o_liga_camera = true;
260         shm->j_sp_velocidade = 10;
261
262         eventos_camera++;
263
264         log_message(
265             "RECONSTRUCAO",
266             "Falha detectada -> câmera acionada e velocidade reduzida"
267         );
268
269         camera.notify_one();
270     }
271 }
272
273     std::lock_guard<std::mutex> lock_camera(mtx_camera);
274     finalizar_camera = true;
275
276     camera.notify_one();
277
278     log_message("RECONSTRUCAO", "Thread finalizada");
279 }
280
281 void coletor_dados(std::mutex &mtx, std::vector <float> &BUFFER){
282
283     log_message(
284         "COLETOR",
285         "Thread inicializada"
286     );
287
288     int idx = -1; // -1 para corrigir o inicio de leitura do vetor
289
290     for (int i = 0; i<20; i++){
291
292         idx++;
293         idx = idx%ELEMENTOS_BUFFERS;
294
295         std::unique_lock<std::mutex> lock (mtx);
296
297         while(dados_nivel == 0){

```

```

298
299         log_message(
300             "COLETOR",
301             "Buffer vazio -> aguardando dados"
302         );
303
304         leitura_buffer_nivel.wait(lock);
305     }
306     //SEÇÃO CRÍTICA
307     log_message(
308         "COLETOR",
309         "Posição lida (nível): " + std::to_string(BUFFER[idx])
310     );
311     dados_nivel--;
312     escrita_buffer_nivel.notify_one();
313     //SEÇÃO CRÍTICA
314
315     lock.unlock();
316
317     //std::this_thread::sleep_for (std::chrono::microseconds(50));
318
319 }
320
321 }
322
323 void inspecao_camera(std::mutex& mtx, MemoriaCompartilhada* shm){
324
325     while(true){
326
327         std::unique_lock<std::mutex> lock(mtx); // Protocolo de entrada
328
329         while(!(eventos_camera > 0 || finalizar_camera)){
330             camera.wait(lock); // Espera até que haja uma falha para
331                 ↳ inspecionar
332         }
333
334         if (eventos_camera == 0 && finalizar_camera) {
335             break;
336         }
337
338         eventos_camera--;
339
340         //inspeciona...
341
342         shm->o_liga_camera = false;
343         shm->e_inspecao = false;
344         lock.unlock(); // Protocolo de saída
345     }
346 }

```

5.5 sincronizacao.hpp

```
1  #ifndef SINCRONIZACAO_H
2  #define SINCRONIZACAO_H
3
4  #include "includes.hpp"
5  #include "shared_memory.hpp"
6
7  #define ELEMENTOS_BUFFERS 10
8
9  void log_message (const std::string& thread, const std::string& mensagem);
10 float numero_aleatorio_debugg();
11
12 /**
13  * @brief Tarefa que recebe comandos do sistema de operação remoto e traduz os
14  * ↳ comandos em setpoint de velocidade
15  * e liga/desliga para o controle de navegação. O comando de navegação que
16  * ↳ deve implementar a lógica de manual/
17  * automático. Exerce a função de ESCRITOR sobre o BUFFER_NAVEGACAO
18  */
19 void comando_navegacao();
20
21 /**
22  * @brief implementação de um controlador PID responsável pelo acionamento dos
23  * ↳ motores (controle de velocidade)
24  * a partir de um setpoint de velocidade recebido pelo Comando de Navegação.
25  * ↳ Exerce a função de LEITOR do BUFFER _NAVEGACAO
26  */
27 void controle_navegacao(std::mutex &mtx, std::vector <float> &BUFFER);
28
29 /**
30  * @brief Registra a distância percorrida fornecida pelo encoder. Possui a
31  * ↳ função de ESCRITOR sobre o BUFFER_NAVEGACAO.
32  */
33 * @param mtx mutex utilizado na variavel compartilhada
34 * @param BUFFER historico de posições
35 */
36 void distancia_percorrida(std::mutex &mtx, std::vector <float> &BUFFER);
37
38 /**
39  * @brief Registra os dados coletados pelo lidar num Banco de Dados. Atua como
40  * ↳ LEITOR do BUFFER_NIVEL.
41  */
42 * @param mtx mutex para sincronizar o buffer compartilhado
43 * @param BUFFER buffer de dados coletados pelo lidar
44 */
45 void coletor_dados(std::mutex &mtx, std::vector <float> &BUFFER);
```



```

45  /**
46   * @brief Recebe os valores do lidar para reconstruir o teto do túnel. Atua
47   *   ↳ como
48   *   ↳ ESCRITOR do BUFFER_NIVEL
49   *   ↳ @param mtx mutex para buffer compartilhado
50   *   ↳ @param BUFFER vetor de dados do nível da distancia do teto
51   */
52 void reconstrucao_teto(std::mutex &mtx_navegacao, std::mutex &mtx_nivel,
53   ↳ std::mutex &mtx_camera, std::vector <float> &BUFFER_NAVEGACAO, std::vector
54   ↳ <float> &BUFFER_NIVEL, MemoriaCompartilhada* shm);
55
56 void inspecao_camera(std::mutex& mtx, MemoriaCompartilhada* shm);
57
58 void operacao_remota(std::mutex &mtx, std::vector <float> &BUFFER);
59
60 #endif

```

5.6 shared_memory.hpp

```

1  #ifndef SHARED_MEMORY_HPP
2  #define SHARED_MEMORY_HPP
3
4  struct MemoriaCompartilhada {
5
6      //sensores e atuadores disponíveis no carrinho:
7      bool i_encoder; //Variável que simula a entrada de um encoder, que troca de
8      ↳ estado a cada metro percorrido pelo robô
9      int i_lidar; //Resposta do sensor LIDAR do veículo exibindo a distância no
10     ↳ eixo y, com relação à altura do robô
11
12     bool o_liga_camera; //Comando para ligar a câmera e realizar fotos da falha
13     ↳ detectada na superfície
14     int o_aceleracao; //Determina a aceleração do veículo em percentual (-100 a
15     ↳ 100%)
16
17     //estados e comandos:
18     bool e_inspecao; //Estado que indica falha detectada na superfície e o robô
19     ↳ está tirando fotos e navegando com velocidade limitada (1:falha, 0: sem
20     ↳ falha)
21     bool e_automatico; //Estado que identifica o modo de operação do robô (0:
22     ↳ manual, 1: automático)
23     bool c_automatico; //Comando para passar o robô para o modo automático
24     ↳ (true). O reset desse comando
25     bool c_man; //Comando para passar o robô para o modo manual (true).
26     int j_sp_velocidade; //Setpoint de velocidade do robô para o controlador de
27     ↳ velocidade.
28 };
29
30 #endif

```

5.7 operacao_remota.cpp

```
1  #include "../include/includes.hpp"
2
3  void operacao_remota(std::mutex& mtx, std::vector<float>& BUFFER){
4      std::unique_lock<std::mutex> lock(mtx); // Protocolo de entrada
5      lock.unlock(); // Protocolo de saída
6  }
```

5.8 Makefile

```
1  TARGET = programa
2
3  CXX = g++
4
5  CXXFLAGS = -std=c++17 -Wall -Wextra -I. -Iinclude -I/opt/homebrew/include
6  ↪ -pthread
7
8  SRCS = main.cpp src/sincronizacao.cpp src/simulacao.cpp
9
10 OBJS = $(SRCS:.cpp=.o)
11
12 all: $(TARGET)
13
14 $(TARGET): $(OBJS)
15     $(CXX) $(CXXFLAGS) $(OBJS) -o $(TARGET)
16
17 %.o: %.cpp
18     $(CXX) $(CXXFLAGS) -c $< -o $@
19
20 run: $(TARGET)
21     ./$$(TARGET)
22
23 clean:
24     rm -f $(OBJS) $(TARGET)
25
26 rebuild: clean all
```